

5002 Final Project

Yirui Tang

December 2025

1 Introduction

This project implements an AI for a triangular variant of Gomoku (five-in-a-row) on an inverted 15-row triangle board. The main decision engine is Monte Carlo Tree Search (MCTS), augmented with a set of strong pattern-based heuristics for both defense and attack. The goal is to produce an AI that plays robustly as either black (first) or white (second), resists common human tactics and opportunistic adversaries, and runs within the course constraints (no deep neural networks; the AI must respond in a few seconds per move).

2 Understanding of MCTS

Monte Carlo Tree Search is a best-first search strategy that builds a search tree using randomized simulations. The algorithm iteratively applies four phases:

- **Selection:** Starting from the root, select child nodes according to a selection policy (commonly UCT) until a node with unexpanded moves or a terminal state is reached. UCT balances exploitation (high average reward) and exploration (low visit count) via an additive term.
- **Expansion:** Expand the selected node by adding one (or more) child nodes representing available moves.
- **Simulation (Rollout):** From the expanded node simulate a full playout to terminal state using a fast policy (often random, but a heuristic-guided policy is better).
- **Backpropagation:** Propagate the simulation result back up the tree, updating visit counts and win statistics for nodes along the path.

Key properties of MCTS:

- As computation increases, the search becomes more focused on promising moves.
- The simulation policy strongly affects playing strength; using heuristics in rollouts typically yields better results than pure random play.
- UCT hyperparameters control the exploration–exploitation tradeoff; practical systems also use domain knowledge to bias the tree or rollout selection.

3 How MCTS is used in this project

3.1 State and action representation

The board is represented as a 15-by-15 array but only a triangular subset is legal. Each move is an (u,v) index. Legal positions are enumerated and used consistently by heuristics and MCTS.

3.2 MCTS implementation details

- **Root and time limit:** For each AI move an MCTS root is constructed from the current board. The search runs within a fixed time budget (5 seconds), which meets the assignment requirement and caps computation per move.
- **Node statistics:** Each node stores visit counts and win totals. During backpropagation a result of draw is treated as a half-win.
- **Selection:** UCT is used (with a tunable exploration constant). Children with zero visits are prioritized to ensure exploration.
- **Expansion:** When a node has untried moves, one is chosen (randomly) and added to the tree.
- **Rollout (simulation) policy:** A heuristic rollout is used: immediate wins/blocks are prioritized; then a neighborhood-based candidate set is scored using adjacency and center-control terms; the top-k candidates are sampled uniformly. Defensive fallbacks ensure the rollout never selects from an empty list, avoiding runtime errors.

3.3 Heuristic integration

MCTS is combined with domain heuristics in a hybrid way:

1. **Fast heuristic checks:** Before running MCTS the AI performs cheap deterministic scans: immediate win detection, immediate opponent win blocking, detection of four-like threats, double open-threes, and two-step kills (TSS).
2. **Strong forced-move handling:** If any forced move (win or critical block) exists, it is played immediately without MCTS, because these moves are deterministic and must be handled first for correctness and speed.
3. **Fallback to MCTS:** Only when heuristics find no forced moves the MCTS routine runs to search for longer-term improvements.

This hybrid strategy yields two benefits: (1) it prevents MCTS from wasting simulations on positions that are already decided by forcing lines, and (2) it ensures the AI does not miss immediate tactical threats that would otherwise cause a loss.

4 From a simple program to active attack

The baseline approach—pure MCTS with naive rollouts—exhibited several weaknesses:

- **Poor opening choices:** pure neighborhood sampling tended to pick corner or peripheral positions in the inverted triangle, which are suboptimal.
- **Weak defenses:** without explicit detection of double threats or two-step kills, the AI could be forced into lost lines by an adversary who creates parallel threats.
- **Stability bugs:** rollout top-k selection could become empty in near-full boards, causing `random.choice` exceptions; GUI highlight and coordinate conversions also introduced bugs.

We addressed these in stages:

1. **Better opening bias:** Introduced a center-gravity (upper-middle for the inverted triangle) preference in both the attack scoring function and the rollout policy. This reduces the chance that the AI starts in an extreme corner or bottom apex.

2. **Pattern-based heuristics:** Implemented detection for:

- immediate wins and immediate blocks,
- four-like threats (4 aligned inside length-5 windows),
- open-three counting (to detect double open-three forks),
- double-threat detection (`creates_double_threat_after_move`),
- two-step kills (`solve_two_step_kill`): simulation of one move and checking whether opponent has any reply that prevents immediate win next move.

These heuristics are tested before MCTS and used inside rollouts where appropriate.

3. **Rollout robustness:** Defensive checks added so that the `rollout_policy` always returns a valid move or a draw indicator; the top-k selection is guarded to never produce an empty list.

4. **GUI correctness:** Fixed highlight drawing logic and coordinate mapping to avoid redrawing all pieces as highlighted; ensured only the last move is highlighted and displayed consistently.

5 Problems encountered and debugging highlights

Although the algorithmic structure looks clean in hindsight, the actual development was a long debugging journey. Some of the most valuable insights came from things that **did not** work at first.

(1) Highlighting gone wild

The very first playable version had a strange visual bug: every time a new stone was played, *all* stones turned into “highlighted mode.” We originally passed a color flag instead of the board matrix into the highlight function, so the line:

```
color = board[u][v]
```

received an integer instead of a matrix → causing both visual corruption and crashes. The fix involved splitting responsibilities clearly: (1) redraw all stones in normal mode, then (2) draw a single red border around only the newest move. This change greatly improved clarity during testing — we could finally see mistakes in real games.

(2) The disappearing candidate move list

Mid-game simulations sometimes crashed with:

```
random.choice([])    # ValueError: list is empty
```

This occurred when the rollout policy filtered candidates too aggressively in late-game, leaving no moves to pick. It only happened in human-vs-AI tests — never in small synthetic testing — which made it harder to reproduce.

We added a defensive two-stage fallback:

1. if top-k scored candidates empty → use full neighbor set
2. if even that is empty → treat as draw (full board)

After that, the entire AI system became much more robust: no more sudden rollouts dying inside MCTS.

(3) When MCTS overthinks and loses instantly

Early versions of AI lost to a very simple double-threat trap: the opponent formed two open-threes far apart, and the AI blocked only one. MCTS *should* theoretically solve this, but with only 5 seconds per move, simulations failed to reach the critical forced line before choosing greedier alternatives.

Our fix: deterministic pattern checks *before* MCTS:

- open-four or four-like $\rightarrow$ win/block immediately
- double open-three & two-step kill $\rightarrow$ detect and enforce forced reply

This turned several embarrassing losses into clean defensive wins.

(4) Copying the entire board every simulation

Rollouts were originally too slow because each simulation created many copies of the 15×15 array. We refactored to reuse local buffers and only copy what changed. Performance improved noticeably, making better use of the 5 second time budget.

(5) Incorrect perspective handling in backpropagation

There were cases where the AI improved the opponent’s chances by miscounting rollout outcomes from the wrong perspective. After adding a consistent “from-parent-player POV” win update rule, MCTS became far more stable — no more moves that obviously helped the opponent.

—

These issues might seem mundane individually, but together they shaped the final AI far more than pure algorithm design. We learned that:

Strong AI in games is not just about smart search — it’s equally about not crashing, not mis-evaluating, and not confusing the GUI.

6 Evaluation and limitations

The hybrid MCTS-plus-heuristics AI shows significantly improved defensive stability and better opening play compared to a naive MCTS implementation with random rollouts. It is much harder for casual human players and baseline automated opponents to “burst” the AI using simple fork tactics.

However, limitations remain:

- Complex long-range sequences can still fool the heuristic detection (pattern enumeration is necessarily approximate).
- The computational budget (5s per move) restricts the depth and number of simulations; parallelization or more efficient state copying would improve performance.
- The heuristics are handcrafted and may not generalize to different board sizes or rule variants without retuning.

7 Division of labour

Yirui Tang:

- Implemented the core MCTS framework, including node data structures, UCT selection, expansion and backpropagation.
- Designed and implemented the rollout policy and its robustness guards.

- Developed pattern-based heuristic modules: immediate win/block, four-like detection, open-three counting, two-step kill detection, and attack scoring.
- Integrated GUI handling (stone drawing, click-to-index mapping) and implemented robust last-move highlighting.
- Performed systematic debugging, testing and iterative improvement of heuristics.

8 Peer Evaluation

Group Member	Score (0-5)
TANG, Yirui	5
MENG, Chuishuo	5
LI, Zixian	5
LIU, Feihao	5